Версии правила v1 и v2

Английский оригинал: RULE_V1_V2.md.

Эта заметка — запись версий для корпусного правила запроса. Она указывает точно, что изменилось между двумя версиями, почему изменение — наименьшее из разрешающих контрпримеры и как инструменты не дают прочитать старый вердикт как текущий.

1. Что такое версия правила

Правило — JSON-документ с `rule_id` (стабильным между версиями) и `rule_version` (приращение при каждом изменении документа). Документ проверяется по `docs/schemas/rule.schema.json` (JSON Schema черновика 2020-12) до использования. Правка правила и его перезагрузка — вот что производит новую версию; подъём номера без изменения документа — no-op, а перезагрузка идентичного текста не помечает предыдущий прогон outdated.

Правило объявляет:

- `scope`: о каком направлении оно говорит (`A_to_B` здесь).
- `framing`: как поток байт режется на сообщения.
- `fields`: смещения, типы и, где известно, ожидаемые значения.
- `hypothesis: true` на поле, чей смысл предполагается, а не наблюдается.

2. Два документа рядом

```
// examples/corpus_rule_v1.json
{
  "schema_version": 1,
  "rule_id": "corpus_request",
  "rule_version": 1,
  "name": "corpus_request_v1",
  "scope": { "direction": "A_to_B" },
  "framing": {
    "type": "length_prefixed", "length_offset": 2, "length_size": 2,
    "byte_order": "big", "length_covers": "payload"
  },
  "fields": [
    { "name": "command", "offset": 0, "type": "uint8", "hypothesis": true, "expected": [1] },
    { "name": "flags", "offset": 1, "type": "uint8", "expected": [0] },
    { "name": "payload_length", "offset": 2, "type": "uint16", "byte_order": "big" },
    { "name": "target", "offset": 4, "type": "enum", "hypothesis": true, "enum": { "...":
16 } },
    { "name": "value", "offset": 5, "type": "uint16", "byte_order": "big",
      "hypothesis": true, "missing_ok": true }
  ]
}
```

v2 — тот же документ с одним изменённым значением:

```
   { "name": "command", "offset": 0, "type": "uint8", "hypothesis": true,
-    "expected": [1] }
+    "expected": [1, 2, 3] }
```

Больше ничего не различается. Фрейминг байт-идентичен, число полей идентично, каждое смещение и тип идентичны. Это намеренно: контрпримеры указывали на одно поле, поэтому изменилось одно поле.

3. Почему это изменение, а не другое

Правило v1 произвело восемьдесят контрпримеров, все на смещении 0, все вида «значение 2 не в ожидаемом [1]» или «значение 3 не в ожидаемом [1]». Были доступны три ответа:

1. Расширить `command` до `[1, 2, 3]`. Движок альтернатив это поддерживает: для `command` он сообщает `entropy_enum` с поддержкой 140 и противоречиями 0, а `constant` (чтение v1) с поддержкой 60 и противоречиями 80. Доказательство говорит, что поле — небольшой замкнутый enum, а не константа.
2. Считать сессии `s2`/`s3` другим правилом и разделить правило на два. Это скрыло бы контрпримеры, не объяснив их; та же раскладка и та же позиция команды получаются во всех трёх сессиях, так что второе правило лишь повторило бы первое с другой константой.
3. Оставить правило как v1 и записать границу. Так поступило бы осторожное чтение, если бы изменение должно было быть догадкой. Это отвергнуто, потому что доказательство за enum прямое, и потому что правило помечает `command` как гипотезу, так что расширение — ожидаемый исход проверки, а не перегиб.

Выбранное изменение — самое узкое из трёх, что убирает контрпримеры, не выдумывая структуру.

4. Различение версий

Прогон CLI по корпусу сообщает отчёты, затем сравниваются две версии:

```
$ python -m src.protocol.cli verify --rule examples/corpus_rule_v1.json \
      --capture .../corpus_capture_01.normalized.json --out /tmp/v1.json
rule corpus_request v1: 280 messages, 80 counterexamples

$ python -m src.protocol.cli verify --rule examples/corpus_rule_v2.json \
      --capture .../corpus_capture_01.normalized.json --out /tmp/v2.json
rule corpus_request v2: 280 messages, 0 counterexamples

$ python -m src.protocol.cli diff --rule-a examples/corpus_rule_v1.json \
      --rule-b examples/corpus_rule_v2.json \
      --report-a /tmp/v1.json --report-b /tmp/v2.json
rule corpus_request: v1 -> v2
  ~ field command.expected: [1] -> [1, 2, 3]
report v1 -> v2
  resolved counterexamples: 80
  introduced counterexamples: 0
  unchanged mismatches: 0
  matched delta: +80
  coverage delta: +0.000
```

Diff правила называет точный путь, который изменился, и оба значения; diff отчёта говорит, что все восемьдесят старых контрпримеров разрешены и ни один не внесён. `matched delta` +80 — это восемьдесят сообщений, перешедших из mismatched в matched. `coverage delta` не изменилось, потому что потоки `B_to_A` остаются `not_applicable` в обеих версиях: область не сдвинулась.

5. Старый вердикт outdated, а не текущий

Когда правило перезагружается с изменением, движок сохраняет предыдущее применение и помечает каждое сообщение в нём `outdated`, с `reason: superseded_by_newer_rule`. `ResultStore` делает то же для хранимых результатов: `mark_outdated(rule)` помечает результаты, принадлежащие более старой версии этого правила. Поэтому отчёт, произведённый v1, показывается как отчёт v1, никогда как текущее состояние, а сравнение версий — единственное место, где числа v1 и v2 ставятся рядом.

6. Что не изменилось

- Раскладка: command, flags, payload_length, target, value на тех же смещениях.
- Фрейминг: с префиксом длины на смещении 2, два байта, big-endian, длина покрывает нагрузку, в обеих версиях.
- Применимость протокола: обе версии ограничены `A_to_B` и не делают утверждений о `B_to_A`.
- Граница `synthetic_live`: обе версии не совпадают там ни с чем и сообщают одно и то же единственное `incomplete`-сообщение. Расширение `command` до значений корпуса не расширило правило до всего мира.

7. Воспроизведение

```
python -m pytest tests/integration/test_reference_report.py \
    tests/hypothesis/test_versioning.py tests/protocol/test_rule.py -q
```